3강. 스프링 부트와 데이터

한국공학대학교
TECH UNIVERSITY OF KOREA

이영곤

» 스타터

- 애플리케이션이 포함해야 하는 의존성을 제공
- 자동 컨피규레이션을 제공
- spring-boot-starter-*
- 스프링부트는 Jar 파일 형태로 독자적인 웹컨테이너에 배포됨
- 마이크로서비스는 DB나 웹컨테이너를 공유해서는 안됨

» main class

- @SpringBootApplication 애노테이션 추가
  ⇒ @Configuration, @EnableAutoConfiguration, @ComponentScan과 동일한 의미
- spring-boot-starter-web 포함

```
@SpringBootApplication
public class Application {
  public static void main(String[] args) {
    SpringApplication.run(Application.class, args);
  }
}
```

» App build

- 앱을 빌드하는 동안 모든 의존성을 실행가능한 Jar(fat Jar)에 포함하는 일을 담당하는 컨피규레이션을 제공해야 함.
- 이는 pom.xml에 정의된 spring-boot-maven-plugin이 수행함
- spring-core, spring-app, spring-context, 스프링부트, 내장형톰캣, logback, Log4j, Slf4j, Jackson lib 등을 포함

```
<plugin>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
```

» 컨피규레이션 파일 사용자 정의

- application.yml을 통해 컨피규레이션 정보 제공

```yaml
server:
  port: ${port:2222}

spring:
  application:
    name: first-service

logging:
  pattern:
    console: "%d{HH:mm:ss.SSS} %-5level %logger{36} - %msg%n"
    file: "%d{HH:mm:ss.SSS} [%thread] %-5level %logger{36} - %msg%n"
#  level:
#    org.springframework.web: DEBUG
  file: app.log

management:
  security:
    enabled: false
```

» Restful web service

- JSON 메시지를 직렬화하고 역직렬화 하는 Jackson 라이브러리는 spring-boot-starter-web에 포함됨.
- 모델 클래스를 선언하기만 하면 됨
- @RestController는 유입되는 HTTP 요청을 컨트럴러 빈 클래스에 설정
- @RequestMapping: 컨트롤러 메서드와 HTTP를 대응하는데 사용
- @GetMapping, @PostMapping, @DeleteMapping = @RequestMapping(method=RequestMethod.GET) ..
- @RequestParam: 요청의 경로와 입력값을 Object로 바인딩
- @RequestBody: 입력 JSON을 잭슨 라이브러리를 사용해 Object로 바인딩

» Spring boot Actuator

- 앱을 모니터링하고 상호작용하기 위한 종단점 제공
  - /beans: 앱에 초기화된 모든 스프링 빈의 목록 표시
  - /env: 스프링의 설정 가능한 환경 속성 목록을 표시
  - /health: 앱의 상태정보 표시
  - /info: 앱의 임의 정보 표시
  - /loggers: 앱의 로거 컨피규레이션 정보 표시
  - /metrics: 앱의 메트릭 정보 표시 (메모리, 스레드, rest 응답시간 등)
  - /trace: 트레이스 정보 표시
- 프로젝트에서 보안 비활성화 하려면 application.yml 파일에

```
management:
```

localhost:2222/metrics


```
{"mem":626192,"mem.free":512993,"processors":8,"instance.uptime":1514270,"uptime":1529231,"systemload.average":-1.0,"heap.committed":569344,"heap.i
onheap.used":56850,"nonheap":0,"threads.peak":35,"threads.daemon":29,"threads.totalStarted":39,"threads":31,"classes":8451,"classes.loaded":8451,"c
,"gc.ps_marksweep.time":130,"httpsessions.max":-1,"httpsessions.active":0,"gauge.response.beans":30.0,"gauge.response.swagger-resources.configurati
resources.configuration.ui":80.0,"gauge.response.webjars.star-star":10.0,"gauge.response.v2.api-docs":133.0,"gauge.response.swagger-resources":13.0
star":3.0,"gauge.response.star-star.favicon.ico":24.0,"counter.status.200.person.id":2,"counter.status.200.swagger-resources.configuration.security
resources":1,"counter.status.200.person":1,"counter.status.500.person.id":2,"counter.services.person.add":1,"counter.status.200.beans":1,"counter.s
star":1,"counter.status.200.swagger-resources.configuration.ui":1,"counter.status.200.v2.api-docs":1,"counter.status.200.webjars.star-star":23}
```

» 개발자 도구

- 클래스경로상의 파일이 변경되면 자동으로 재시작
- spring-boot-devtools 의존성 포함하면 됨
- 변경시 재시작 되지 않도록 하려면 제외 항목에 포함

```
<dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-devtools</artifactId>
        <optional>true</optional>
</dependency>
```

```
spring:
  application:
    name: first-service
  devtools:
    restart:
      exclude: static/**
```

» 개발자 도구

- 스프링클라우드는 원격서버에서 컨피규레이션 가져옴. 이를 위해 부트스트랩 컨택스트에 App내 필요한 최소 설정, 즉 App 이름, 컨피규레이션 주소 설정 등을 해야 함. 즉 컨피규레이션을 외부 소스에서 읽어 올 수 있도록.
- 부트스트랩 컨택스트는 bootstrap.yml을 사용
- 부트스트랩 비활성화 하려면 spring.cloud.bootstrap.enabled를 false로 설정
- 부트스트랩 컨피규레이션 파일 이름은 spring.cloud.bootstrap.name에서 변경 가능. 위치는 location으로 변경

```yaml
spring:
  application:
    name: first-service
  cloud:
    config:
      uri: http://192.168.99.100:8888
```

» 엔티티

- 엔티티는 DB 테이블과 매핑되는 객체 의미

» 엔티티 매니저

- 엔티티를 관리해 DB와 App 사이에서 객체를 생성, 수정, 삭제하는 역할 수행
- 엔티티 매니저를 만드는 곳이 엔티티 매니저 팩토리
- 스프링부트 내부에서 엔티티 매니저 팩토리 하나만 생성해서 관리하고 @PersistenceContext 혹은 @Autowired를 사용해 엔티티 매니저 사용

» 영속성 컨택스트

- 엔티티 매니저는 엔티티를 영속성 컨택스트에 저장
- 영속성 컨택스트는 엔티티를 관리하는 가상의 공간
- 1차 캐시, 쓰기 지연, 변경 감지, 지연 로딩 특징 있음

» 1차 캐시

- 캐시의 키는 엔티티의 @Id가 달린 식별자이며 값은 엔티티
- 엔티티 조회시 1차 캐시에서 조회하고 값이 있으면 반환, 없으면 DB에서 읽어와 캐시에 저장하고 반환

» 쓰기 지연

- 트랜잭션을 커밋하기 전까지 DB에 질의문 보내지 않고 쿼리를 모았다가 커밋시 쿼리를 한번에 실행

» 변경 감지

- 트랜잭션 커밋시 1차 캐시에 저장되어 있는 엔티티의 값과 현재 엔티티의 값을 비교해 변경된 값이 있으면 변경사항 감지해 DB에 자동으로 반영

» 지연 로딩

- 쿼리로 요청한 데이터를 App에 바로 로딩하지 않고 필요할 때 쿼리를 보내 데이터를 조회하는 것 의미

» 정리

- DB 접근을 최소화해 성능을 높이기 위함

» 스프링 데이터

- 비즈니스 로직에 집중할 수 있도록 DB 사용기능을 클래스 레벨에서 추상화함
- 스프링 데이터에서 제공하는 인터페이스를 통해 스프링 데이터 사용

» 스프링 데이터 JPA

- 스프링 데이터의 공통적인 기능에서 JPA의 유용한 기술 추가
- 리포지터리 역할을 하는 인터페이스를 만들어 DB 테이블 조회, 수정, 생성, 삭제 작업을 간단히 수행

```
@Repository
public interface BookRepository extends CrudRepository<Book, Integer> {
}
```

» 스프링 데이터 JPA

```
@Service
public class BookService {

  @Autowired
  BookRepository bookRepository;

  public void test() {
    bookRepository.save(new Book(5, "my book","yklee", new java.util.Date()));
    List<Book> bookList = (List<Book>) bookRepository.findAll();
    bookRepository.deleteById(5);
  }

}
```

» Entity 샘플 코드

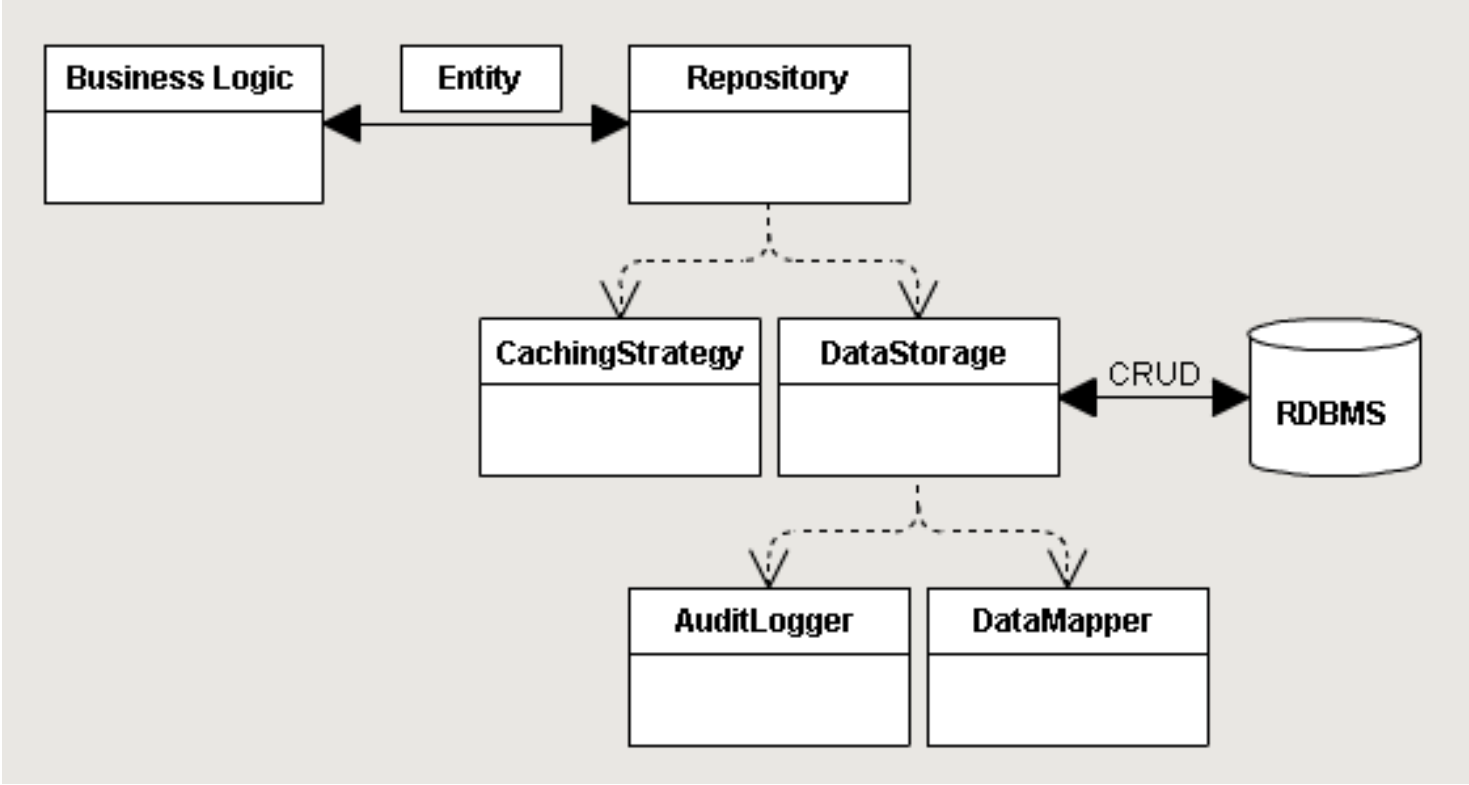
```java
@Entity
@Getter
@NoArgsConstructor(access= AccessLevel.PROTECTED)
@AllArgsConstructor
public class Book {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id", updatable= false)
    private long id;

    @Column(name = "name", nullable = false)
    private String name;

    @Column(name = "author")
    private String author;

    @CreationTimestamp
    @Column(name = "published", columnDefinition =
    "TIMESTAMP DEFAULT CURRENT_TIMESTAMP", updatable = false)
    @Temporal(TemporalType.TIMESTAMP)
    private Date published;
}
```

» Entity sample



» Example

```
@Document(collection = "person")
public class Person {

@Id
private String id;
private String firstName;
private String lastName;
private int age;
private String gender;


public interface PersonRepository extends
MongoRepository<Person, String> {

  public List<Person> findByLastName(String lastName);
  public List<Person> findByAgeGreaterThan(int age);

}
```

» Example of application.properties

```
spring.application.name=second
spring.datasource.url=jdbc:mysql://mysql:3306/web?useSSL=false&server
Timezone=UTC&characterEncoding=UTF-8
spring.datasource.username=root
spring.datasource.password=rootpassword
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.database-platform=org.hibernate.dialect.MySQL8Dialect
# Thymeleaf 캐시 비활성화 (개발 시 사용)
spring.thymeleaf.cache=false
server.port=8090
```

클라우드 네이티브 개발에 적합한 이유

- 경량 + 빠른 실행 (Lightweight Editor)
  - 기존 IDE(예: IntelliJ IDEA, Eclipse IDE) 대비 메모리 사용량과 로딩 속도가 훨씬 낮음
  - 특히 컨테이너, 클라우드 개발 환경에서 빠른 재시작/접속이 중요해지면서 큰 장점
- 쿠버네티스, MSA 환경에서는 무거운 IDE보다 VS Code가 훨씬 효율적

클라우드 네이티브 개발에 적합한 이유

- 확장성 중심 아키텍처 (Extension-based)
- 기본은 가볍고, 필요한 기능만 추가하는 구조
- 대표 확장:
  - Python extension for VS Code
  - Spring Boot Extension Pack
  - Docker extension for VS Code
  - Kubernetes extension for VS Code
- 즉, “IDE + DevOps + Cloud Tool”까지 하나로 통합 가능

클라우드 네이티브 개발에 적합한 이유

- 클라우드 / 컨테이너 친화적
- Remote 개발 지원:
  - SSH 접속
  - 컨테이너 내부 개발
  - WSL 연동
- DevContainer, Codespaces 지원 → 환경 일관성 확보

클라우드 네이티브 개발에 적합한 이유

- Git 통합 (DevOps 친화)
  - 기본적으로 Git UI 내장
  - Git 작업:commit / push / diff / branch 관리
  - GitHub, GitLab과 자연스럽게 연결

설치할 Extension pack (Ctrl+Shift+X)

- Extension pack for Java
- Docker
- Spring boot extension pack
- Container tools

감사합니다

THANK YOU